SQL

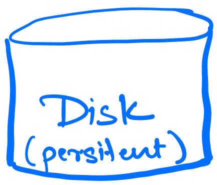
Structured Query Language

Databases

SQL NoSQL

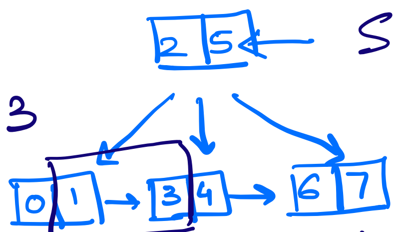
Relational Database Management System (RDBMS)

- SQL is the major component and it is used to access data stored in RDBMS



RDMS

Data Structure used for RDBMS is
B+ Tree



m = 3

Split will be m-1 = 2

Access is easy

Data is stored in the leaf nodes in sorted order and there is link

∴ The Reason RDBMS uses B+ Tree (M Tree) data structure is becoz it reduces the size of the Tree as it increases the number of children unlike the BST (where there are only 2 childs) so this will help minimize the Read/Write ops

Example :- Phonebook where we want to find the phone number for a user the way B+Tree works is it uses some index in this case it is the name and tree will store it in alphabetical order so that search is easy to the leaf node That will have the no. corresponding to the name

RDBMS

(There are a bunch of tables
And we create it using a SQL)

People Table

Phone Numbers	Name
123-45-678	Alice
123-46-789	Bob

Schema of People

SQL

```
Create Table People (
    PhoneNumber int,
    Name varchar(100)
);
```

* Schema - helps us to specify how the data should look like within the Table

People Table

Phone Numbers	Name
123-45-678	Alice
123-46-789	Bob

Homes Table

Phone	Address

* People & Homes are related meaning these 2 tables can have a relationship where we have constraints like every phone number associated to a house has to belong to a person (This can be achieved using the Foreign key constraint of SQL) or unique Phone Number should be unique (Primary key constraint) or JOINS eg:-

```
Select People.name, Home.address
From People, Homes
Where People.phone = Homes.phone;
```

※ RDBMS are majorly ACID complaint

A - Atomicity :- Like the 1 Transaction example which is All or Nothing meaning if sys crashes before commit nothing sored but after commit than that 1 Transacth is saved

C - Consistency :- Data consistency basically is following the constraints/rules defined for RDBMS (FK, PK, Not null etc)

I - Isolation :- Multiple Transaction happening concurrently on a DB, want " to happen in order & not affect each other Issues eg:- Dirty Reads or Phantom Reads. Isolation does Serialization of transaction

D - Durability :- Disk is eg of durability however Redis / Cache though faster stores everything in memory which is not durable

1 Transaction - Begin

1
2
3

commit

Insert
Select
Update
Delete

In case commit done and sys crashes its fine but if sys crashes before commit than the transaction not persited